



INTERNAL — MAY 2026

Building an Agentic Chatbot Suite with Claude

How we used Claude to design, build, and ship three integrated libraries.

HOW WE PROCEEDED

Built with Claude, end to end

Claude was used as a development partner across every phase — from the phased roadmap to the final library code, with built-in review and verification.

1. Plan

A phased roadmap (Phase 0 → 12) was generated and refined with Claude before any code.

2. Code

Components, providers, framework adapters, and tests were drafted with Claude across Python, React, and Angular.

3. Review

Specialized Claude subagents review each change — Python tests, MCP compliance, React/Angular component patterns.

4. Verify

Claude Code hooks auto-run ruff, typecheck, and build on every edit — so generated code is never assumed correct.

CAPABILITIES

Not a chatbot — an agent platform

The library goes well beyond message-in / message-out. It is built around an agent loop that can call tools, chain them, and reach external systems safely.

- **Multi-LLM**

Anthropic Claude, OpenAI, Azure OpenAI, and the long tail via LiteLLM — one interface.

- **MCP-native**

Stdio, SSE, and HTTP transports. External tool servers plug in without code changes.

- **Multimodal**

Text, images, and file attachments are first-class parts of the protocol.

- **Tools as first-class**

Python callables, HTTP endpoints, OpenAPI specs, MCP servers — all in one registry.

- **Per-user auth**

Tools receive scoped OAuth tokens and API keys through the user context — no shared credentials.

- **Skills & prompts**

Reusable prompt and skill bundles, discovered from disk and surfaced to the agent.

INTEGRATION

Three lines, three stacks

Any team can drop the chatbot into their existing stack. The frontend speaks to the same Python backend over a stable HTTP + SSE protocol.

Python backend

```
from chatbot.integrations
    .fastapi import mount

mount(app, agent=my_agent)
```

FastAPI · Flask · Django · Starlette

React frontend

```
<ChatbotProvider
  config={cfg}>
  <FloatingChatbot />
</ChatbotProvider>
```

React 17 · 18 · npm: chatbot-react

Angular frontend

```
<chatbot-floating
  [config]="cfg" />

// standalone component
```

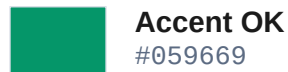
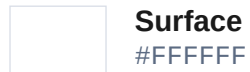
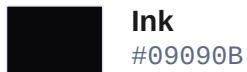
Angular 17 · npm: chatbot-angular

.NET integration · coming next

A design system, also built with Claude

Tokens, components, light + dark modes — designed and documented with Claude, then translated into Tailwind + CSS variables shared by React and Angular.

Design tokens



Components & themes

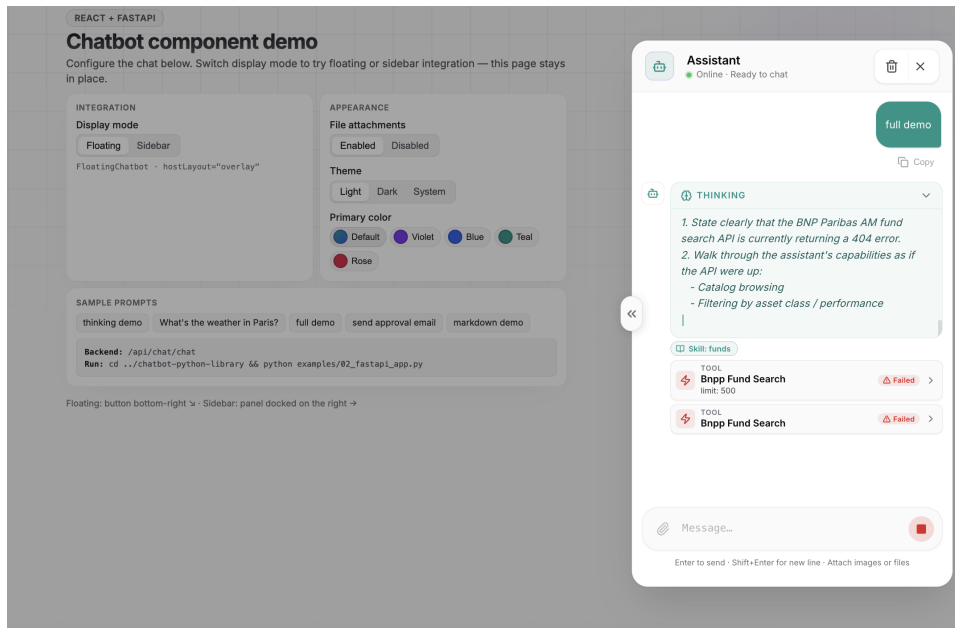
- **20+ components**
Floating chatbot, message bubbles, streaming cursor, tool-call cards, markdown + code rendering.
- **Light & dark modes**
Tokens defined once, themed via CSS variables — apps inherit their host site's mode.
- **Accessibility-aware**
Keyboard-first composer, ARIA labels on interactive elements, focus management in the floating panel.
- **Documented**
Full design-system reference shipped as HTML + PDF inside the React library.

PLUG & PLAY

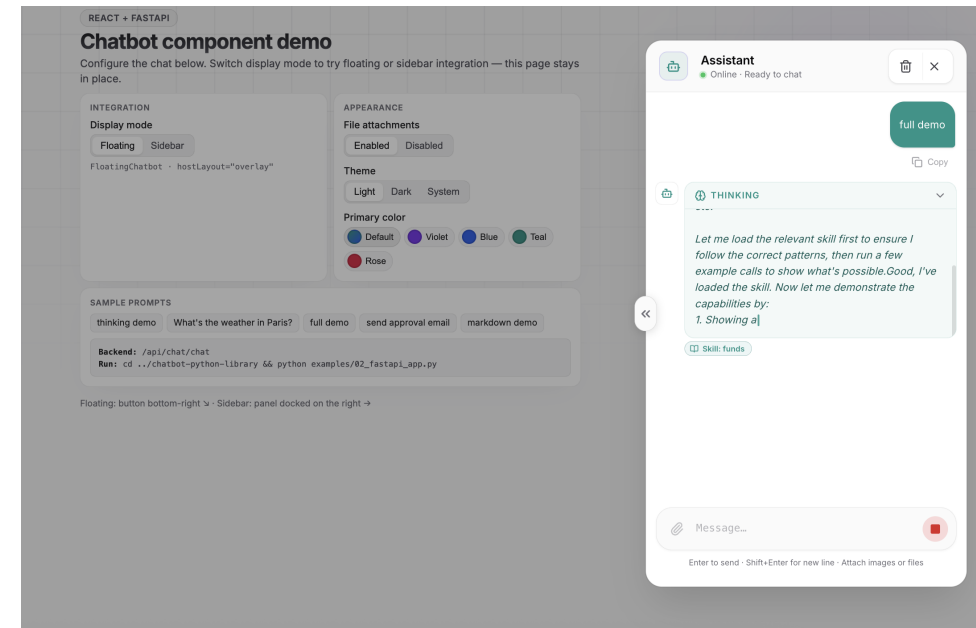
Same look and feel — across stacks

The React and Angular libraries render the exact same UI from the same design tokens and the same wire protocol. One product experience, every frontend.

Angular 17 · chatbot-angular



React · chatbot-react



One backend. One design system. Drop-in for any React or Angular app.

Tested, gated, automated

Each library has its own test suite, runs in CI on every PR, and ships to its registry through a tag-driven publish workflow.

Python

PyPI · chatbot

TESTS

15 pytest files

Core, providers, MCP, tools, storage, FastAPI integration.

CI

ruff + pytest on every PR

GitHub Actions · ci.yml

RELEASE

Tag python-v* → PyPI

Trusted publishing (no token).

React

npm · chatbot-react

TESTS

Vitest + Testing Library

Store, transport, utils, theme — jsdom-backed.

CI

Typecheck + tests + build

GitHub Actions · ci.yml

RELEASE

Tag react-v* → npm

npm publish --provenance.

Angular

npm · chatbot-angular

TESTS

Karma + Jasmine

Service, SSE transport, utils, component smoke.

CI

ng test + ng build

Headless Chrome on PRs.

RELEASE

Tag angular-v* → npm

Provenance, dedup-gated release event.